show a typical json schema structure for a mongodb database, along with some example fields it may contain. vary the examples you give, for example show how enums, arrays, strings, integers, floats are stored in json schema

A MongoDB JSON Schema (used for document validation) typically defines:

- The document type (`bsonType`)
- Required fields
- Field types
- Constraints (e.g., enum values, ranges, patterns)
- Nested objects and arrays

Example:

JSON

```
{
  "$jsonSchema": {
    "bsonType": "object",
    "required": [
      "user_id",
      "name",
      "email",
      "status"
    ],
    "properties": {
      "_id": {
        "bsonType": "objectId",
        "description": "MongoDB document ID"
      },

      "user_id": {
        "bsonType": "int",
        "minimum": 1,
        "description": "Unique user identifier"
      },

      "name": {
        "bsonType": "string",
        "minLength": 2,
        "maxLength": 100
      },

      "email": {
        "bsonType": "string",
        "pattern": "^.+@.+\\..+$"
      },

      "status": {
        "enum": [
          "active",
```

```
      "inactive",
      "suspended"
    ]
  },

  "age": {
    "bsonType": "int",
    "minimum": 0,
    "maximum": 150
  },

  "salary": {
    "bsonType": "double",
    "minimum": 0
  },

  "is_verified": {
    "bsonType": "bool"
  },

  "created_at": {
    "bsonType": "date"
  },

  "tags": {
    "bsonType": "array",
    "items": {
      "bsonType": "string"
    }
  },

  "scores": {
    "bsonType": "array",
    "items": {
      "bsonType": "double"
```

```
        }
      },

      "address": {
        "bsonType": "object",
        "properties": {
          "street": {
            "bsonType": "string"
          },
          "city": {
            "bsonType": "string"
          },
          "postcode": {
            "bsonType": "string"
          }
        }
      },

      "preferences": {
        "bsonType": "object",
        "properties": {
          "theme": {
            "enum": [
              "light",
              "dark"
            ]
          },
          "notifications": {
            "bsonType": "bool"
          }
        }
      }
    }
  }
}
```

An example document that conforms to this schema:

JSON

```
{
  "_id": "6842c9e6f4b4c5d9a8f12345",
  "user_id": 1001,
  "name": "Alice Tan",
  "email": "alice@example.com",
  "status": "active",
  "age": 28,
  "salary": 5500.75,
  "is_verified": true,
  "created_at": "2026-06-06T12:00:00Z",

  "tags": [
    "premium",
    "researcher",
    "beta-user"
  ],

  "scores": [
    95.5,
    88.0,
    91.25
  ],

  "address": {
    "street": "123 Jalan Teknologi",
    "city": "Johor Bahru",
    "postcode": "81300"
  },

  "preferences": {
    "theme": "dark",
    "notifications": true
  }
}
```

Common MongoDB JSON Schema field types:

Type	Example
`string`	`"name": "John"`
`int`	`"age": 25`
`long`	`"population": 3200000000`
`double`	`"price": 19.99`
`decimal`	`"tax": 0.075`
`bool`	`"active": true`
`date`	`"created_at": ISODate(...)`
`objectId`	`"_id": ObjectId(...)`
`array`	`"tags": ["A","B"]`
`object`	`"address": {...}`
`enum`	`"status": "active"` from a predefined list
`null`	`"middle_name": null`

A more complex example from a legal corpus might look like:

JSON

```
{
  "$jsonSchema": {
    "bsonType": "object",
    "required": [
      "act_id",
      "section_id",
      "section_category"
    ],
    "properties": {
      "act_id": {
        "bsonType": "string"
      },

      "act_title": {
        "bsonType": "string"
      },

      "section_id": {
        "bsonType": "string"
      },

      "section_title": {
        "bsonType": "string"
      },

      "full_section": {
        "bsonType": "string"
      },

      "section_category": {
        "enum": [
          "offence",
          "procedural",
          "administration_of_justice",
          "evidence",
```

```
    "definition"
  ]
},

"keywords": {
  "bsonType": "array",
  "items": {
    "bsonType": "string"
  }
},

"confidence_score": {
  "bsonType": "double",
  "minimum": 0,
  "maximum": 1
},

"citations": {
  "bsonType": "array",
  "items": {
    "bsonType": "object",
    "properties": {
      "target_section": {
        "bsonType": "string"
      },
      "citation_type": {
        "enum": [
          "reference",
          "amendment",
          "exception"
        ]
      }
    }
  }
}
```

```
      }
    }
  }
```

This demonstrates most of the structures commonly encountered in MongoDB schemas: strings, integers, floating-point numbers, booleans, enums, arrays, nested objects, and arrays of objects.

You're encountering a distinction between **JSON Schema** and **MongoDB's implementation of JSON Schema**.

`type` **vs** `bsonType`

Standard JSON Schema
Uses `type`:

JSON

```
{
  "type": "object",
  "properties": {
    "age": {
      "type": "integer"
    }
  }
}
```

Valid JSON Schema types include:

```
string
number
integer
boolean
object
array
null
```

MongoDB JSON Schema Validation

MongoDB extends JSON Schema with `bsonType` because MongoDB stores BSON, not pure JSON.

JSON

```
{
  "$jsonSchema": {
    "bsonType": "object",
    "properties": {
      "age": {
        "bsonType": "int"
      }
    }
  }
}
```

MongoDB-specific BSON types include:

```
objectId
int
long
double
decimal
date
bool
string
array
object
null
```

Does `double` exist in JSON?

No.

In standard JSON there is only one numeric type:

JSON

```
{
  "integer_value": 42,
  "floating_value": 3.14159
}
```

Both are simply numbers.

A JSON Schema would therefore use:

JSON

```
{
  "type": "number"
}
```

or

JSON

```
{
  "type": "integer"
}
```

There is no separate `float`, `double`, or `decimal` in JSON.

Why does MongoDB have `double`, `int`, and `decimal`?

MongoDB's underlying BSON format distinguishes them:

BSON Type	Example
`int`	`42`
`long`	`9007199254740991`
`double`	`3.14159`
`decimal`	`Decimal128("123.45")`

This allows MongoDB to preserve storage type and precision.

For example:

JavaScript

```
{
  age: NumberInt(42),
  salary: 5500.75,
  tax_rate: NumberDecimal("0.075")
}
```

These are different BSON types internally.

Which should you use?

If you are:

Documenting your database structure in a thesis/report
Usually use generic JSON types:

JSON

```
{
  "act_id": "613",
  "confidence_score": 0.95,
  "keywords": ["money laundering"]
}
```

or

JSON

```
{
  "act_id": {
    "type": "string"
  },
  "confidence_score": {
    "type": "number"
  }
}
```

This is easier for readers.

Creating an actual MongoDB validator

Use `bsonType`:

JSON

```
{
  "$jsonSchema": {
    "bsonType": "object",
    "properties": {
      "confidence_score": {
        "bsonType": "double"
      }
    }
  }
}
```

A useful rule of thumb:

- `type` → standard JSON Schema, portable across systems.
- `bsonType` → MongoDB validation schema, MongoDB-specific.
- `double` is a BSON type, not a JSON type.
- `number` is a JSON type, encompassing both integers and floating-point values.

why am i getting an error on this schema? is this the incorrect way to reference a previous variable?

```
"enrolment_target": {
            "type": "integer",
            "minimum": 30,
            "maximum": 2000
        },
        "enrolled_count": {
            "type": "integer",
            "minimum": 0,
```

Show more

The issue is that **JSON Schema does not allow most validation keywords (such as `maximum`) to reference another field's value**.

This is invalid in standard JSON Schema:

JSON

```
{
  "maximum": {
    "$data": "enrolment_target"
  }
}
```

because `maximum` expects a numeric literal:

JSON

```
{
  "maximum": 2000
}
```

not an object.

Why have you seen `$data` before?

`$data` is **not part of the JSON Schema specification**. It is an extension implemented by some validators, most notably **Ajv** (Another JSON Schema Validator).

With Ajv and `$data` support enabled, you would write:

JSON

```
{
  "enrolled_count": {
    "type": "integer",
    "minimum": 0,
    "maximum": {
      "$data": "/enrolment_target"
    }
  }
}
```

Notice the JSON Pointer syntax (`"/enrolment_target"`), not simply `"enrolment_target"` .

If you're using MongoDB

MongoDB's `$jsonSchema` validator **does not support** `$data` **references**.

You cannot express:

```
enrolled_count <= enrolment_target
```

using MongoDB JSON Schema alone.

Instead, you would need one of:

Option 1: Application-level validation
Validate before inserting/updating:

JavaScript

```
if (enrolled_count > enrolment_target) {
    throw new Error("Enrolled count exceeds target");
}
```

Option 2: MongoDB validator using `$expr`

MongoDB supports document-level expressions outside `$jsonSchema`:

JavaScript

```
db.createCollection("courses", {
  validator: {
    $expr: {
      $lte: [
        "$enrolled_count",
        "$enrolment_target"
      ]
    }
  }
})
```

This is the preferred MongoDB solution.

Summary

Feature	Supported?
`"maximum": 2000`	✅ Standard JSON Schema
`"maximum": {"$data": ...}`	❌ Standard JSON Schema
`$data` in Ajv	✅ With plugin/support enabled
`$data` in MongoDB `$jsonSchema`	❌

Feature	Supported?
`enrolled_count <= enrolment_target` in MongoDB	✅ Using `$expr`

If you tell me **what validator/database/framework you're using** (MongoDB, Mongoose, Ajv, Pydantic, FastAPI, etc.), I can show the correct syntax for that specific environment.

this is just simple json for a schema. does this mean i should just set maximum to a literal number?

Yes.

If you're defining a simple JSON Schema, then `maximum` should be a literal numeric value:

JSON

```
{
  "enrolment_target": {
    "type": "integer",
    "minimum": 30,
    "maximum": 2000
  },
  "enrolled_count": {
    "type": "integer",
    "minimum": 0,
    "maximum": 2000
  }
}
```

JSON Schema is primarily for validating the structure and constraints of individual fields. It is not designed to express relationships between fields such as:

```
enrolled_count <= enrolment_target
```

unless you're using advanced features supported by a specific validator.

If your goal is merely to document the schema (e.g., for a database design document, thesis, API specification, or data dictionary), then it is perfectly acceptable to use a fixed maximum:

JSON

```
{
  "enrolled_count": {
    "type": "integer",
    "minimum": 0
  }
}
```

or

JSON

```
{
  "enrolled_count": {
    "type": "integer",
    "minimum": 0,
    "maximum": 2000
  }
}
```

The business rule that `enrolled_count` should never exceed `enrolment_target` would typically be documented separately, for example:

Field	Type	Constraints
enrolment_target	integer	30–2000
enrolled_count	integer	0–2000; must not exceed `enrolment_target`

This separation is common because cross-field constraints are often enforced by application logic or database rules rather than by basic JSON Schema definitions.